thws Technische Hochschule Würzburg-Schweinfurt

Projektarbeit

# INTERAKTIVE LERNPLATTFORM FÜR GRUNDLAGEN INFORMATIK

M. Nöth, R. Schröder, E. Ebert, D. T. Pham, K. Popov

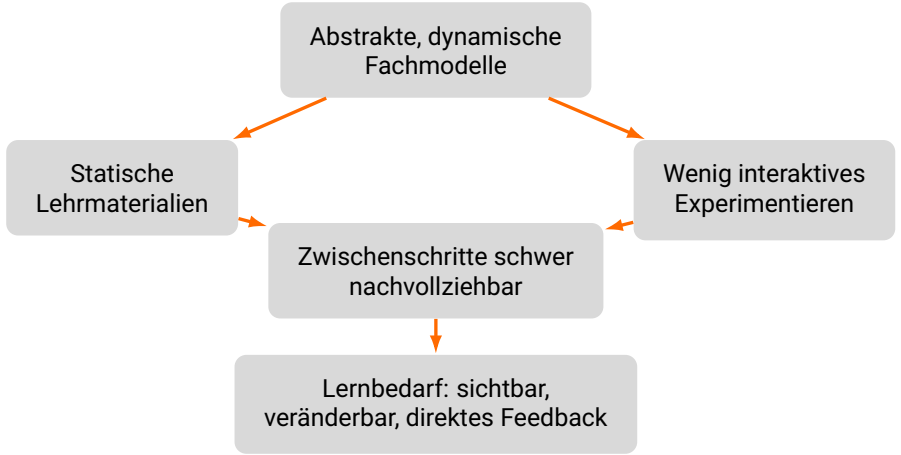


thws
Projektarbeit
INTERAKTIVE LERNPLATTFORM FÜR GRUNDLAGEN INFORMATIK
M. Nöth, R. Schröder, E. Ebert, D. T. Pham, K. Popov

**Person: Enrico Zeit: Intro**

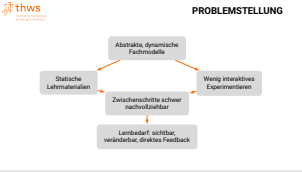
# 1. KONTEXT UND ZIELE

# PROBLEMSTELLUNG








**Person: Enrico**
**Zeit: 3 bis 5 min**

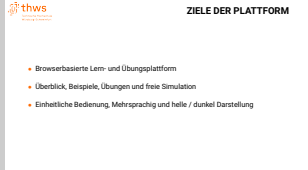
- test
- test
- test

# ZIELE DER PLATTFORM

- Browserbasierte Lern- und Übungsplattform
- Überblick, Beispiele, Übungen und freie Simulation
- Einheitliche Bedienung, Mehrsprachig und helle / dunkel Darstellung








**Person: Enrico**
**Zeit: 1 bis 2 min**

- test
- test
- test

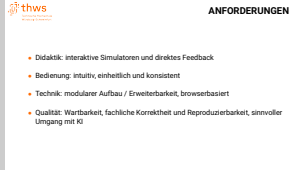
# 2. ANFORDERUNGEN UND PROJEKTABLAUF

# ANFORDERUNGEN

- Didaktik: interaktive Simulatoren und direktes Feedback
- Bedienung: intuitiv, einheitlich und konsistent
- Technik: modularer Aufbau / Erweiterbarkeit, browserbasiert
- Qualität: Wartbarkeit, fachliche Korrektheit und Reproduzierbarkeit, sinnvoller Umgang mit KI








**Person: Markus**
**Zeit: 2 min**

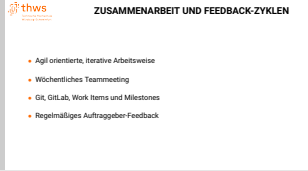
- test
- test
- test

# ZUSAMMENARBEIT UND FEEDBACK-ZYKLEN

- Agil orientierte, iterative Arbeitsweise
- Wöchentliches Teammeeting
- Git, GitLab, Work Items und Milestones
- Regelmäßiges Auftraggeber-Feedback








**Person: Markus**
**Zeit: 1 bis 2 min**

- test
- test
- test

# 3. LÖSUNGSANSATZ: ARCHITEKTUR UND DESIGN

# TECHNISCHE BASIS

- React 19 + TypeScript + React Router
- Vite + Tailwind CSS 4 + shadcn/ui
- Zustand + i18next
- Monaco + OpenJDK javac + TeaVM





thws
TECHNISCHE BASIS
- React 19 + TypeScript + React Router
- Vite + Tailwind CSS 4 + shadcn/ui
- Zustand + i18next
- Monaco + OpenJDK javac + TeaVM

**Person: Tan (Enrico)**
**Zeit: 1 min**

- test
- test
- test

# MODULARE PLATTFORMARCHITEKTUR

- Fachlich getrennte Topics
- Gemeinsame Shared Components
- `moduleCatalog.tsx` als zentrale Modulverwaltung





thws
MODULARE PLATTFORMARCHITEKTUR
- Fachlich getrennte Topics
- Gemeinsame Shared Components
- moduleCatalog.tsx als zentrale Modulverwaltung

**Person: Tan (Enrico)**
**Zeit: 1 min**

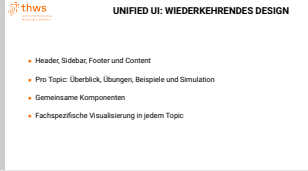
- test
- test
- test

# UNIFIED UI: WIEDERKEHRENDES DESIGN

- Header, Sidebar, Footer und Content
- Pro Topic: Überblick, Übungen, Beispiele und Simulation
- Gemeinsame Komponenten
- Fachspezifische Visualisierung in jedem Topic



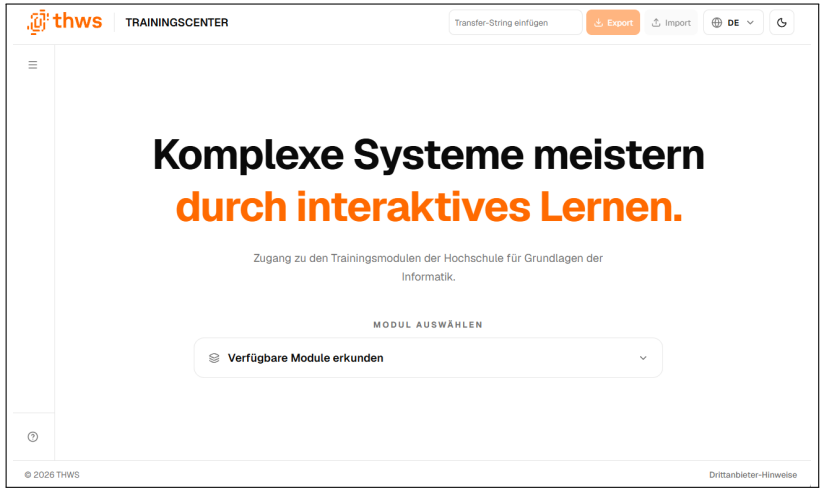





**Person: Tan (Enrico)**
**Zeit: 1 min**

- test
- test
- test

# UI – MAINPAGE





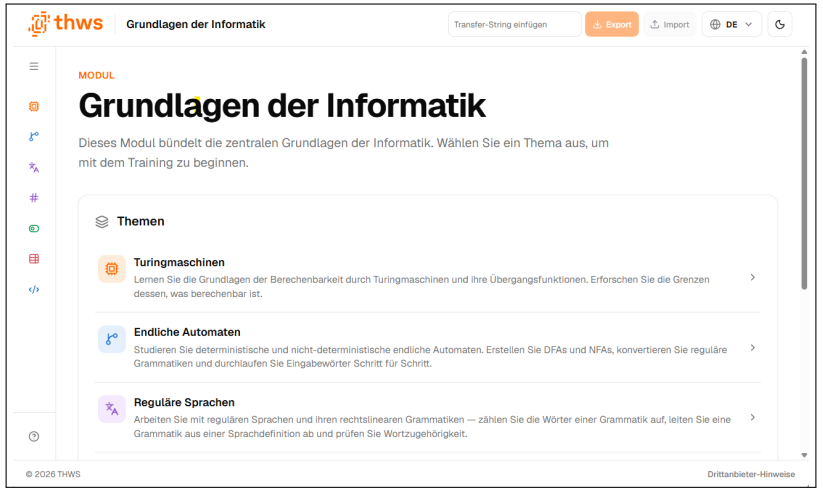






**Person: Tan Zeit: Nur kurz zeigen –> 0 min**

# UI – MODULPAGE





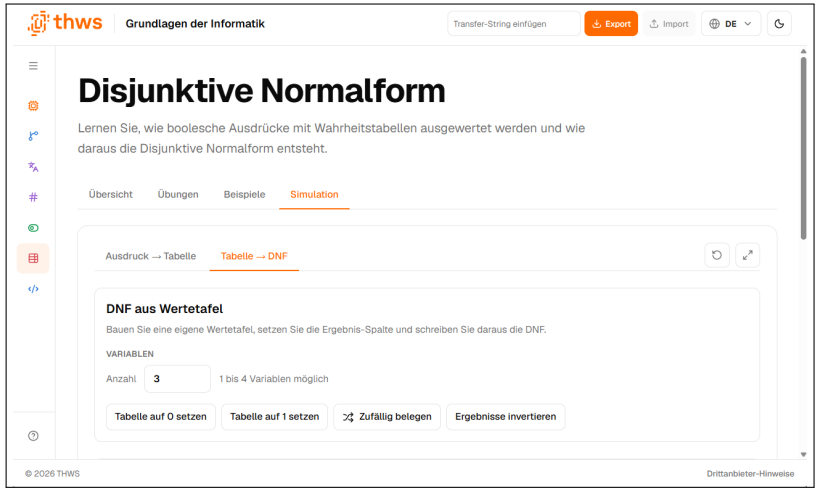






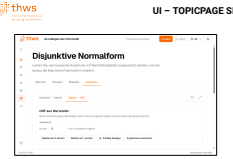
**Person: Tan Zeit: Nur kurz zeigen –> 0 min**

# UI – TOPICPAGE SIMULATOR



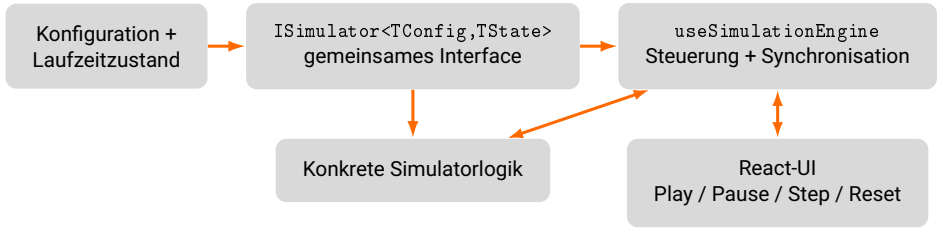




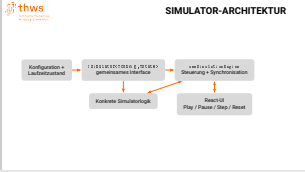

**Person: Tan Zeit: Nur kurz zeigen –> 0 min**

# SIMULATOR-ARCHITEKTUR








**Person: Richard**
**Zeit: 2 bis 3 min**

- test
- test
- test

# IMPORT, EXPORT UND LOKALE PERSISTENZ

- Gemeinsamer Transfer-Codec
- Kompakte Base64-Transferstrings
- Local Storage








**Person: Markus**
**Zeit: 1 bis 2 min**

- test
- test
- test

# 4. KI-EINSATZ IM PROJEKT

# KI-WERKZEUGE IM PROJEKT

- Stitch, ChatGPT, Gemini, Claude, Codex, Antigravity, verschiedene OpenCode und lokale Modelle
- MCP, LSP, Graphify und Agent Skills



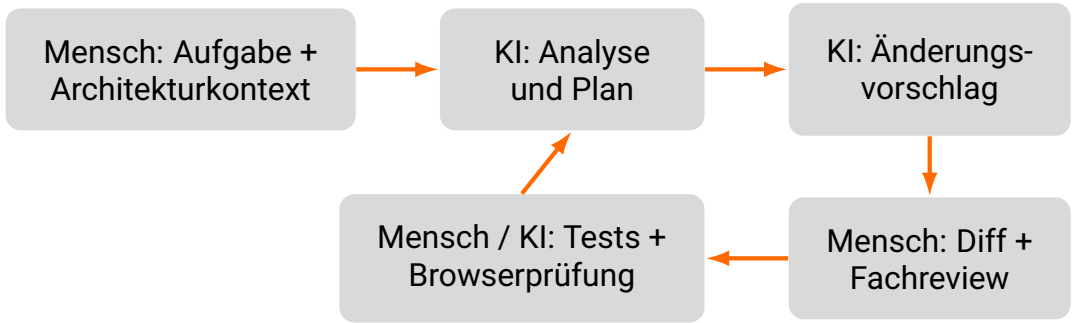


thws
KI-WERKZEUGE IM PROJEKT
- Stitch, ChatGPT, Gemini, Claude, Codex, Antigravity, verschiedene OpenCode und lokale Modelle
- MCP, LSP, Graphify und Agent Skills

**Person: Kristian**
**Zeit: 1 min**

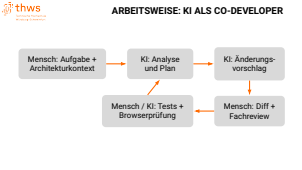
- test
- test
- test

# ARBEITSWEISE: KI ALS CO-DEVELOPER








**Person: Kristian**
**Zeit: 2 min**

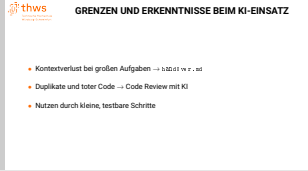
- test
- test
- test

# GRENZEN UND ERKENNTNISSE BEIM KI-EINSATZ

- Kontextverlust bei großen Aufgaben → `handover.md`
- Duplikate und toter Code → Code Review mit KI
- Nutzen durch kleine, testbare Schritte








**Person: Markus**
**Zeit: 2 min**

- test
- test
- test

# 5. QUALITÄT UND DEPLOYMENT-STRATEGIE

# QUALITÄTSSICHERUNG

- ESLint + Prettier + fallow
- Vitest
- Vite Build + Playwright + MCP
- Lighthouse + manuelles Review








**Person: Tan (Markus)**
**Zeit: 1 min**

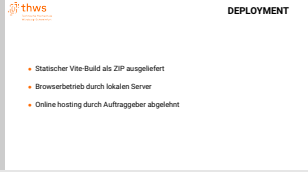
- test
- test
- test

# DEPLOYMENT

- Statischer Vite-Build als ZIP ausgeliefert
- Browserbetrieb durch lokalen Server
- Online hosting durch Auftraggeber abgelehnt








**Person: Markus**
**Zeit: 1 min**

- test
- test
- test

# FACHMODUL: TURINGMASCHINE

# TURINGMASCHINE KONFIGURIEREN

- Übergänge als $(q', b, D)$ in der Übergangstabelle
- Zustände, Haltezustände und Anfangsband definierbar
- Eingabe- und Bandalphabet werden automatisch abgeleitet
- Partielle Übergangsfunktionen: Leere Zellen führen zum Halt



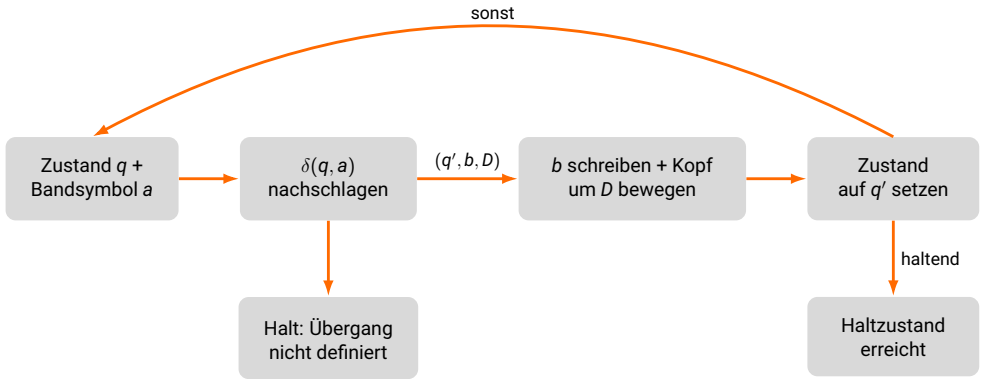


thws
TURINGMASCHINE KONFIGURIEREN
- Übergänge als $(q', b, D)$ in der Übergangstabelle
- Zustände, Haltezustände und Anfangsband definierbar
- Eingabe- und Bandalphabet werden automatisch abgeleitet
- Partielle Übergangsfunktionen: Leere Zellen führen zum Halt

**Person: Richard**
**Zeit: 1 min**

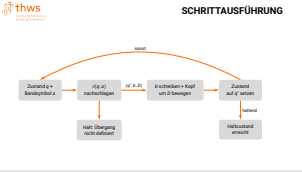
- test
- test
- test

# SCHRITTAUSFÜHRUNG







**Person: Richard**
**Zeit: 1 min**

- test
- test
- test

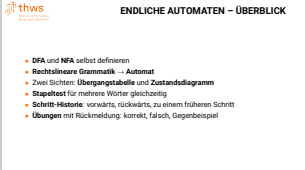
# FACHMODUL: ENDLICHE AUTOMATEN

# ENDLICHE AUTOMATEN – ÜBERBLICK

- **DFA** und **NFA** selbst definieren
- **Rechtslineare Grammatik** → **Automat**
- Zwei Sichten: **Übergangstabelle** und **Zustandsdiagramm**
- **Stapeltest** für mehrere Wörter gleichzeitig
- **Schritt-Historie**: vorwärts, rückwärts, zu einem früheren Schritt
- **Übungen** mit Rückmeldung: korrekt, falsch, Gegenbeispiel



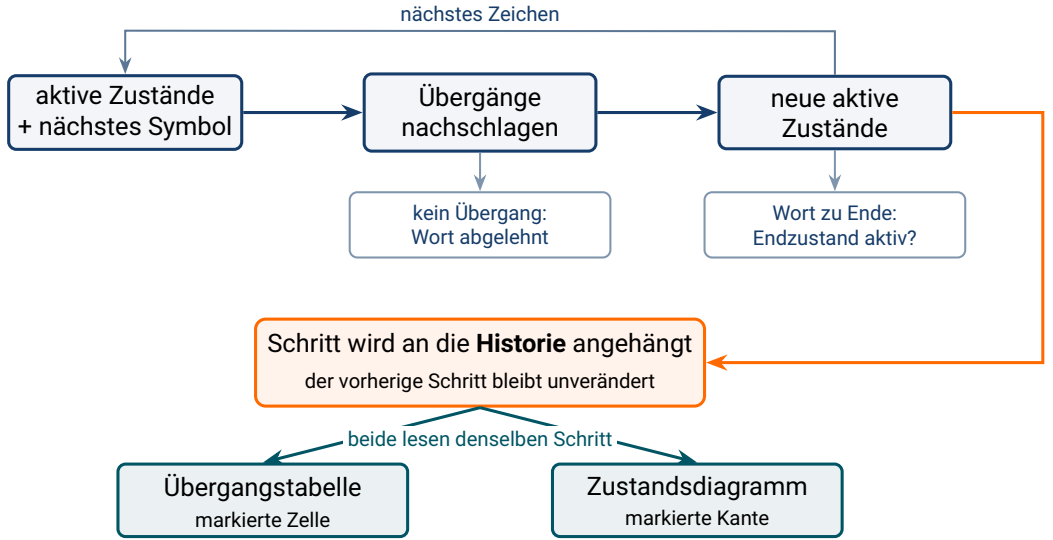





**Person: 1,5 min**
**Zeit: min**

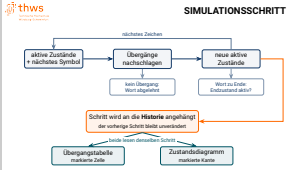
- test
- test
- test

# SIMULATIONSSCHRITT







**Person: Kristian**
**Zeit: 1 min**

- Aktive Zustände + Symbol bestimmen den nächsten Schritt
- Fehlender Übergang → Wort sofort abgelehnt
- Am Wortende entscheidet der Endzustand
- Simulationsschritte werden gespeichert → Zurückspringen möglich
- Tabelle und Diagramm nutzen denselben Zustand → immer synchron

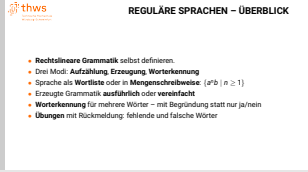
# FACHMODUL: REGULÄRE SPRACHEN

# REGULÄRE SPRACHEN – ÜBERBLICK

- **Rechtslineare Grammatik** selbst definieren.
- Drei Modi: **Aufzählung**, **Erzeugung**, **Worterkennung**
- Sprache als **Wortliste** oder in **Mengenschreibweise**: $\{a^n b \mid n \geq 1\}$
- Erzeugte Grammatik **ausführlich** oder **vereinfacht**
- **Worterkennung** für mehrere Wörter – mit Begründung statt nur ja/nein
- **Übungen** mit Rückmeldung: fehlende und falsche Wörter



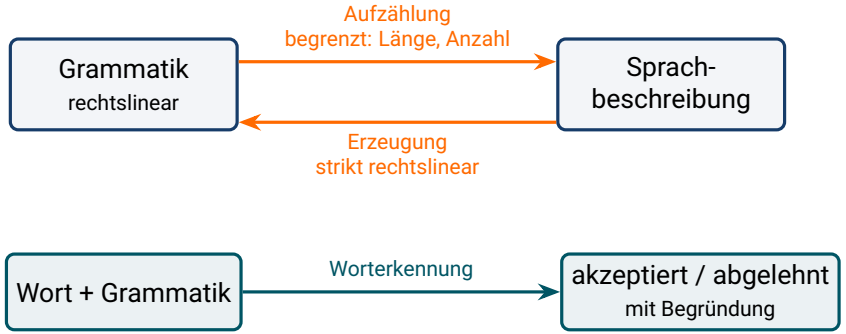





**Person:**
**Zeit: 1 min**

- Einstiegsfolie des zweiten Themas, parallel zum Überblick bei den Automaten.
- Nur benennen, was die Studierenden tun können – die Begründungen kommen auf den
- folgenden Folien.
- Die drei Modi heißen in der Oberfläche "Grammatik -> Sprache", "Sprache -> Grammatik"
- und "Wort erkennen"; die nächste Folie vertieft sie.
- Ausführlich/Vereinfacht: ein Nichtterminal pro Präfix gegenüber zusammengefassten
- Nichtterminalen mit denselben Fortsetzungen – rechtslinear bleiben beide.
- Überleitung: "Diese drei Arbeitsweisen im Zusammenhang."

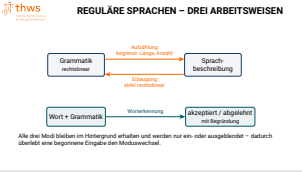
# REGULÄRE SPRACHEN – DREI ARBEITSWEISEN




Alle drei Modi bleiben im Hintergrund erhalten und werden nur ein- oder ausgeblendet – dadurch überlebt eine begonnene Eingabe den Moduswechsel.







**Person:**
**Zeit: 1 min**

- Kurz durch die drei Modi gehen, nicht vertiefen.
- Der Satz unten ist eine UI-Entscheidung: Komponenten bleiben gemountet, damit Eingaben nicht verloren gehen.
- Überleitung zur nächsten Folie: "Nicht jede Eingabe wird akzeptiert – und das mit Absicht."

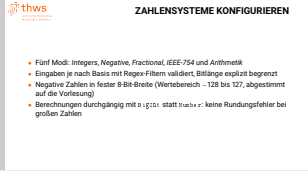
# FACHMODUL: ZAHLENSYSTEME

# ZAHLENSYSTEME KONFIGURIEREN

- Fünf Modi: *Integers*, *Negative*, *Fractional*, *IEEE-754* und *Arithmetik*
- Eingaben je nach Basis mit Regex-Filtern validiert, Bitlänge explizit begrenzt
- Negative Zahlen in fester 8-Bit-Breite (Wertebereich −128 bis 127, abgestimmt auf die Vorlesung)
- Berechnungen durchgängig mit `BigInt` statt `Number`: keine Rundungsfehler bei großen Zahlen



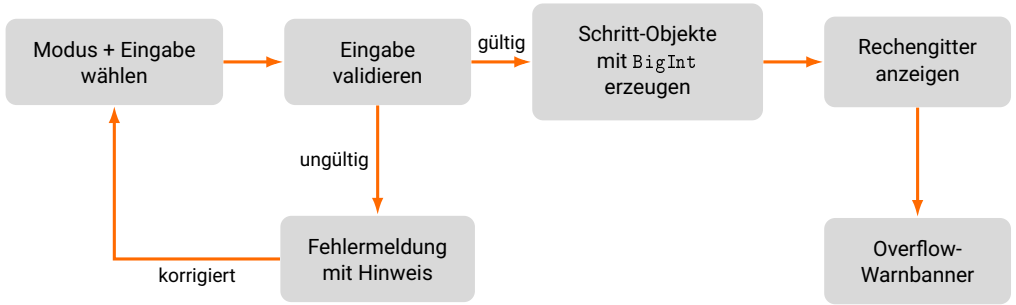





**Person: Enrico**
**Zeit: 1 min**

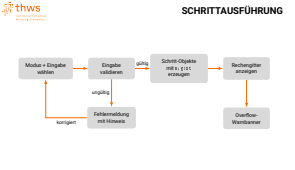
- Einstieg: Das Modul deckt den Zahlenbereich der Vorlesung ab – von der Basisumrechnung
- bis zur Gleitkommadarstellung.
- Die BigInt-Entscheidung kurz begründen: Number verliert ab 2^53 an Präzision.
- Überleitung: "Wie läuft so eine Simulation konkret ab?"

# SCHRITTAUSFÜHRUNG








**Person: Enrico**
**Zeit: 1 min**

- Hier zählt der Ablauf in der App, nicht das Rechenverfahren selbst.
- Jede Stufe ist einzeln aufklappbar und mit i18n-Schlüsseln verknüpft –
- dadurch bleibt die Erklärung bei Sprachwechsel konsistent.
- Beim Überlauf: das Banner warnt nur, korrigiert aber nicht – bewusste didaktische Entscheidung.

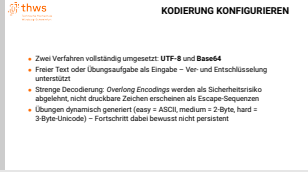
# FACHMODUL: KODIERUNG

# KODIERUNG KONFIGURIEREN

- Zwei Verfahren vollständig umgesetzt: **UTF-8** und **Base64**
- Freier Text oder Übungsaufgabe als Eingabe – Ver- und Entschlüsselung unterstützt
- Strenge Decodierung: *Overlong Encodings* werden als Sicherheitsrisiko abgelehnt, nicht druckbare Zeichen erscheinen als Escape-Sequenzen
- Übungen dynamisch generiert (easy = ASCII, medium = 2-Byte, hard = 3-Byte-Unicode) – Fortschritt dabei bewusst nicht persistent



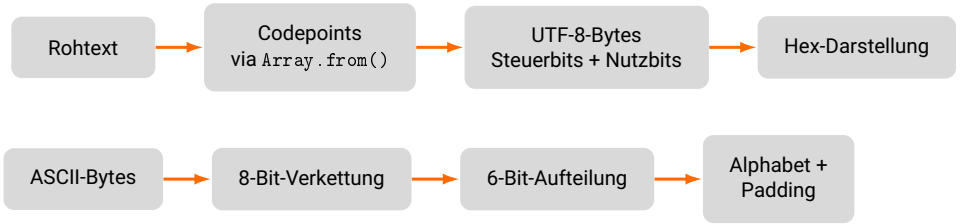





**Person: Enrico**
**Zeit: 1 min**

- Einstieg: Kodierung kam später dazu, auf Wunsch des Auftraggebers –
- im Lastenheft war sie nicht vorgesehen.
- Besonderheit gegenüber anderen Modulen: Übungen sind ephemer und werden
- nicht dauerhaft gespeichert, der freie Simulator dagegen schon.
- Überleitung: "Wie sieht die Transformation im Detail aus?"

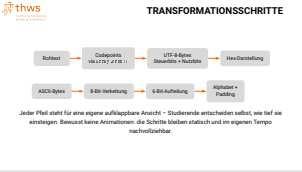
# TRANSFORMATIONSSCHRITTE




Jeder Pfeil steht für eine eigene aufklappbare Ansicht – Studierende entscheiden selbst, wie tief sie einsteigen. Bewusst keine Animationen: die Schritte bleiben statisch und im eigenen Tempo nachvollziehbar.








**Person: Enrico**
**Zeit: 1 min**

- Zwei Pipelines explizit getrennt zeigen: dieselbe didaktische Struktur,
- aber unterschiedliche fachliche Stufen.
- Bei UTF-8 die `Array.from()`-Entscheidung erwähnen: sichere Iteration
- über Codepoints vermeidet Surrogate-Pair-Fehler bei Emojis.
- Rückfrage: "Warum keine Animation?" – weil kontrollierbares Tempo
- didaktisch mehr bringt als automatisch ablaufende Schritte.

# FACHMODUL: DISJUNKTIVE NORMALFORM

# DISJUNKTIVE NORMALFORM – ÜBERBLICK

- Zwei Aufgabenmodi: Wahrheitstabelle nach DNF und umgekehrt
- Drei Stufen: kanonische DNF, minimale DNF und Minimalform
- Minimierung mit Quine-McCluskey und Branch-and-Bound
- Semantische Prüfung erkennt logische Äquivalenz
- Begrenzung auf maximal 4 Variablen für exakte Synthese



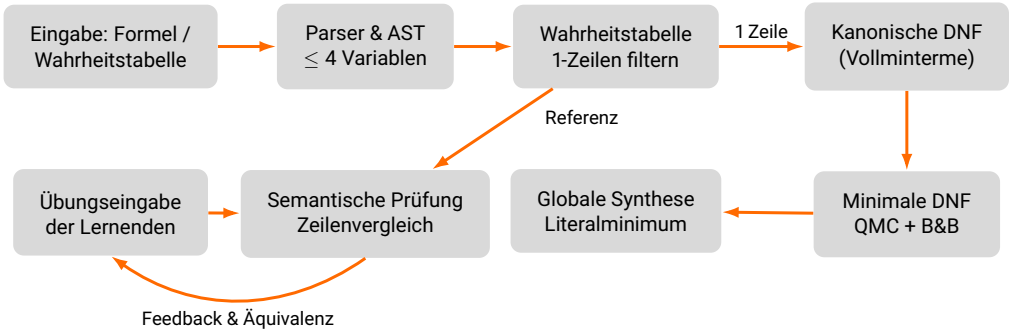


thws
DISJUNKTIVE NORMALFORM – ÜBERBLICK
- Zwei Aufgabenmodi: Wahrheitstabelle nach DNF und umgekehrt
- Drei Stufen: kanonische DNF, minimale DNF und Minimalform
- Minimierung mit Quine-McCluskey und Branch-and-Bound
- Semantische Prüfung erkennt logische Äquivalenz
- Begrenzung auf maximal 4 Variablen für exakte Synthese

**Person:**
**Zeit: min**

-

# SCHRITTAUSFÜHRUNG









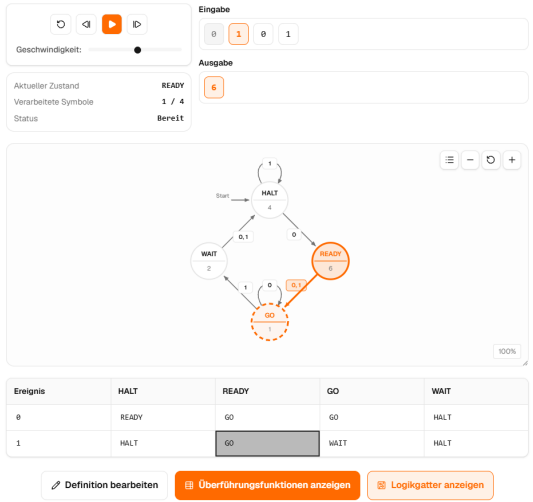


**Person:**
**Zeit: min**

-

# FACHMODUL: MOORE MEALY

# INTERAKTIVE SIMULATOR DARSTELLUNG




- Automaten frei konfigurieren
- Eingaben Schritt für Schritt simulierbar
- Aktueller Zustand und Übergang direkt sichtbar
- Vor- und Zurückspringen in der Simulation








**Person: Markus**
**Zeit: 0.75 min**

-

# AUTOMAT ANALYSIEREN & SCHALTUNG DARSTELLEN

**Überführungsfunktion**

- Zustandsübergänge übersichtlich darstellen
- Ausgabe abhängig von Moore bzw. Mealy

**Wahrheitstabelle**

- Zustände und Eingaben binär codieren
- Boolesche Funktionen automatisch ableiten
- DNF minimieren

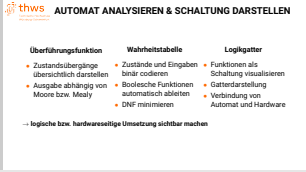
**Logikgatter**

- Funktionen als Schaltung visualisieren
- Gatterdarstellung
- Verbindung von Automat und Hardware

→ **logische bzw. hardwareseitige Umsetzung sichtbar machen**





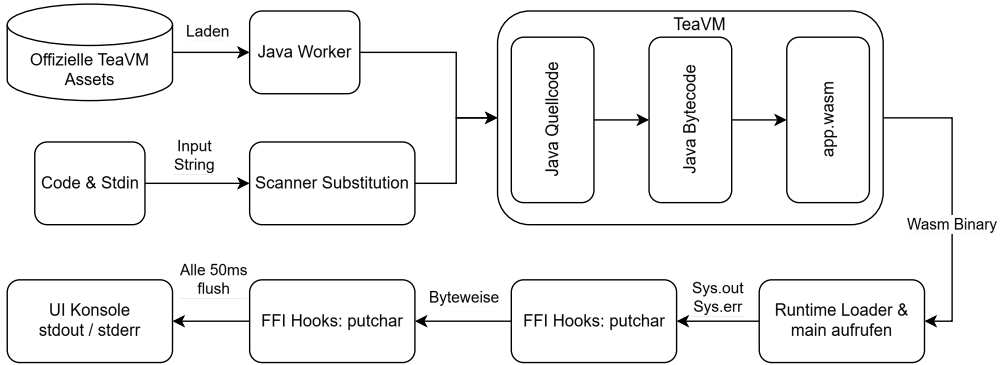



**Person: Markus**
**Zeit: 0.75 min**

-

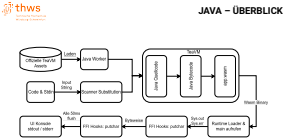
# FACHMODUL: JAVA

# JAVA – ÜBERBLICK







**Person: Markus**
**Zeit: 1 min**

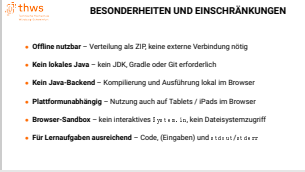
-

# BESONDERHEITEN UND EINSCHRÄNKUNGEN

- **Offline nutzbar** – Verteilung als ZIP, keine externe Verbindung nötig
- **Kein lokales Java** – kein JDK, Gradle oder Git erforderlich
- **Kein Java-Backend** – Kompilierung und Ausführung lokal im Browser
- **Plattformunabhängig** – Nutzung auch auf Tablets / iPads im Browser
- **Browser-Sandbox** – kein interaktives `System.in`, kein Dateisystemzugriff
- **Für Lernaufgaben ausreichend** – Code, (Eingaben) und `stdout/stderr`








**Person: Markus**
**Zeit: 0.5 min**

-

# 6. INTERAKTIVE LIVE-DEMO

# DEMO-ZIEL: DEN LERNWEG SICHTBAR MACHEN

- Eingabe verändern
- Schrittweise ausführen
- Zwischenschritte erklären





thws
DEMO-ZIEL: DEN LERNWEG SICHTBAR MACHEN
- Eingabe verändern
- Schrittweise ausführen
- Zwischenschritte erklären

**Person: 3 min Tan DNF | 2 min Richard TM**
**Zeit: 5min**

- test
- test
- test

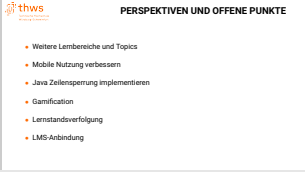
# 7. AUSBLICK

# PERSPEKTIVEN UND OFFENE PUNKTE

- Weitere Lernbereiche und Topics
- Mobile Nutzung verbessern
- Java Zeilensperrung implementieren
- Gamification
- Lernstandsverfolgung
- LMS-Anbindung








**Person: Kristian**
**Zeit: 2 min**

- test
- test
- test